

EPSS-TPG: end-to-end model architecture

1. Raw CVE record

Description text plus, optionally, one or more LLM summary columns. Source-CSV format (Sec4A4Aec, Megavul) or NVD JSON.

Example: "A buffer overflow in OpenSSL 1.0.1f allows a remote attacker to ..."

2. spaCy linguistic frontend

en_core.web.sm

Five-step build, one paragraph at a time:

- tokenise + POS-tag + lemmatise + dependency-parse
- named-entity recognition (PERSON, ORG, GPE, ...)
- sentence segmentation
- noun-chunk detection
- character offsets preserved per token

3. TPG construction: 13 node types, 13 base edge types

One DOCUMENT root. For each paragraph: one PARAGRAPH node, each sentence becomes a SENTENCE, each spaCy token becomes a TOKEN. Spans become ENTITY / NOUN-PHRASE / VERB-PHRASE / CLAUSE nodes; content verbs become PREDICATE nodes; their dependents become ARGUMENT nodes with SRL roles inferred from the dep label.

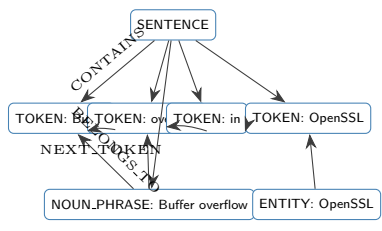
Edges are wired as the nodes are created:

CONTAINS DOC→PARA→SENT→(TOKEN, ENTITY, NP, VP, CLAUSE, PRED)
BELONGS_TO ENTITY/NP/VP/CLAUSE/PRED/ARG → TOKEN
DEP TOKEN (head) → TOKEN (child), dep label stored on edge
NEXT_TOKEN adjacent TOKEN→TOKEN, intra and inter sentence
NEXT_SENT adjacent SENT→SENT (document order)
NEXT_PARA adjacent PARA→PARA
SRL_ARG TOKEN/ARG → PRED, role label (ARG0, ARG1, ARGM-*)
COREF, RST_RELATION, DISCOURSE, added later by enrichment passes
ENTITY_REL, SIMILARITY, AMR_EDGE (CoreferencePass, DiscoursePass, ...)

feeds

Example sub-graph

For "Buffer overflow in OpenSSL 1.0.1f"



4. Security frontend: regex + curated dictionaries

Adds 12 security node types on top of the linguistic graph (frontend), then 10 typed SEC_* edges (SecurityRelationsPass).

Rule sources:

CVE-\d{4}-\d{4} → ENTITY tagged CVE_ID
CWE-\d{1,4} → ENTITY tagged CWE_ID
\b\d+\.\d+(\.\d+)*\b → ENTITY tagged VERSION (CVSS scores filtered)
CVSS:3\.\d/... → ENTITY tagged SEVERITY
C-style function regex → ENTITY tagged CODE_ELEMENT
SOFTWARE dictionary (~300 names) → ENTITY tagged SOFTWARE
ATTACK VECTOR dict → ENTITY tagged ATTACK_VECTOR
IMPACT dict (rce, dos, escalation, ...) → ENTITY tagged IMPACT
VULN TYPE dict (overflow, XSS, ...) → ENTITY tagged VULN_TYPE
REMEDIATION regex set → ENTITY tagged REMEDIATION

SecurityRelationsPass then emits cartesian-product edges:

SEC_AFFECTS (CVE→SW), SEC_HAS_VERSION (SW→VER), SEC_CLASSIFIED_AS (CVE→CWE),
SEC_LOCATED_IN (VULN→CODE), SEC_EXPLOITED_BY (CVE→AV), SEC_CAUSES (AV→IMPACT),
SEC_MITIGATED_BY (CVE→REM), SEC_USES_FUNCTION (CVE→CODE), SEC_THREATENS (AV→SW),
SEC_HAS_SEVERITY (CVE→SEV). Active only when --include-security-edges is on; edge-type vocabulary then grows from 13 to 23.

5. SecBERT embedding pass joins to spaCy nodes

jackadama/SecBERT,

frozen, 768-dim

Three encodings, all from the same SecBERT forward pass:

- SENTENCE nodes get the [CLS] embedding of the sentence text.
- TOKEN nodes get the **character-span-aligned** mean of WordPiece subtoken vectors.
- ENTITY and NOUN-PHRASE nodes get a [CLS] embedding of their text (if not already set).

Alignment between WordPiece subtokens and spaCy tokens:

- SecBERT tokenises sentence text with return_offsets_mapping=True.
- For each spaCy TOKEN node, find the subtoken indices whose char span overlaps the token's char span.
- aligned = subword_embeddings[matching].mean(dim=0), stored at node.properties.extra["embedding"].
- Nodes without text get the zero vector.

6. PyG tensor construction (per CVE graph)

The exporter walks the TPG once and produces:

x : $N \times 781$ = [node-type one-hot (13)] || [SecBERT 768-dim] (784 with --add-source-labels)
edge_index : $2 \times M$ source→target indices over all kept edges
edge_type : M integer in $[0, R)$ where $R = 13$ or $R = 23$ (base + SEC_*)
edge_attr : $M \times R$ one-hot of edge_type
 y : scalar soft EPSS in $[0, 1]$ or binary KEV label
tabular: 55-dim or 57-dim built only when --hybrid is set

7. Multi-view GGNN with node-conditioned attention fusion

Input projection $H^{(0)} = \text{Dropout}(\text{GELU}(\text{LayerNorm}(W_{\text{in}} X)))$, $H^{(0)} \in \mathbb{R}^{N \times h}$, $h = 128$.

Edges are partitioned into V views:

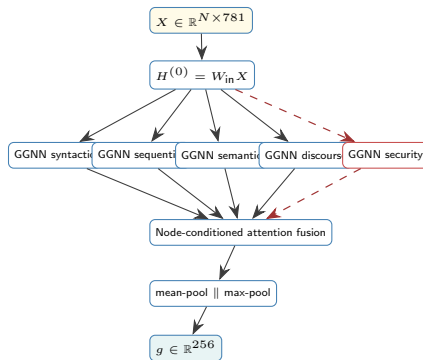
syntactic DEP, CONTAINS, BELONGS_TO
sequential NEXT_TOKEN, NEXT_SENT, NEXT_PARA
semantic COREF, SRL_ARG, AMR_EDGE
discourse RST_RELATION, DISCOURSE, ENTITY_REL, SIMILARITY
security all SEC_* (only when security edges are enabled)

Per view, a Gated Graph Neural Network (Li et al., 2015): $H_v = \text{ViewEncoder}_v(H^{(0)}, E_v)$ runs $K = 3$ message-passing steps over that view's edges, with a residual, LayerNorm and Dropout.

Node-conditioned fusion (per node i): a query q_i from $H_i^{(0)}$ and per-view keys k_v^i from $H_v[i, :]$ produce scaled-dot-product scores $q_i \cdot k_v^i / \sqrt{h_i}$; softmax over V gives weights α_i^v ; the fused embedding is $h_i = \sum_v \alpha_i^v H_v[i, :]$.

detailed view

Multi-view backbone



Solid = always on. Dashed red = security view, active only with --include-security-edges.

8. Tabular branch (hybrid mode only)

Per-CVE feature vector tabular (55-dim default, 57 when EPSS is kept as input):

- CVSS v3 score (1) + has-CVSS flag (1)
- CVSS v3 vector components (4+2+3+2+2+3+3+3 = 22 one-hot)
- Top-25 CWE multi-hot + "other" bucket (26)
- CWE count, references count, vulnerability age, public-exploit flag, exploit count (1 each)
- EPSS score, EPSS percentile (optional, +2)

Tabular MLP: Linear(TAB, 128) → BN → ReLU → Drop →

Linear(128, 64) → ReLU → Drop.

Output: $t \in \mathbb{R}^{64}$.

9. Fusion head and prediction

Inputs: graph vector $g \in \mathbb{R}^{256}$ from stage 7 and tabular embedding $t \in \mathbb{R}^{64}$ from stage 8.

Concatenate, then a 3-layer MLP with batch norm and dropout in the middle:

$[g; t] \in \mathbb{R}^{320} \rightarrow \text{Linear}(320, 128) \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{Drop} \rightarrow \text{Linear}(128, 64) \rightarrow \text{ReLU} \rightarrow \text{Drop} \rightarrow \text{Linear}(64, 1)$

Output logit $\hat{\ell} \in \mathbb{R}$; predicted probability $\hat{p} = \sigma(\hat{\ell}) \in [0, 1]$.

Training: BCE-with-logits against the label y , with a positive-class weight w^+ = negatives/positives so the rare high-EPSS / KEV class is not drowned out. Optimiser: AdamW, lr = 10^{-3} , weight decay = 10^{-4} , grad clip 1.0. ReduceLROnPlateau on val PR-AUC, early stop after 15 epochs without gain.

Inference: $\hat{p}$ is the predicted exploitation probability; risk tier (LOW/MEDIUM/HIGH/CRITICAL) is read off the operational thresholds 0.1, 0.4, 0.7.